

Enterprise Infrastructure Provisioning & Orchestration Guide

Target Operating System: Ubuntu 22.04 LTS / 24.04 LTS (AWS EC2 Instance)

Project Integration: SkillPulse 3-Tier Web Application Stack

Author: Mayank Rawal

Purpose: Implementation of Local Containers, Multi-Node Kubernetes Clustering, and GitOps Automation Engine

Table of Contents

1. Host System Environment Synchronization
 2. Docker Engine Architecture & Post-Install Setup
 3. Kubernetes-in-Docker (KinD) Binary Provisioning
 4. Kubectl CLI Command Utilities Setup
 5. Declarative Cluster Bootstrapping with Ingress Forwarding
 6. Registering the GitHub Actions Self-Hosted Daemon Service
 7. Final System Integrity Validation
-

1. Host System Environment Synchronization

Before deploying nested runtimes, clean up older database stubs, synchronize package manifests, and upgrade active Linux headers to patch kernel-level container layers.

```
# Update localized repository package pointers
sudo apt-get update -y

# Perform kernel/package upgrades to guarantee network interface stability
sudo apt-get upgrade -y

# Install baseline automation tools and transport layer wrappers
sudo apt-get install -y curl wget apt-transport-https ca-certificates gnupg lsb-release git
```

2. Docker Engine Architecture & Post-Install Setup

KinD maps cluster control planes inside container sandboxes; hence, a stable, persistent host-level Docker engine is mandatory.

Step 2.1: Implement Cryptographic Trust (Docker GPG Signatures)

Bash

```
sudo mkdir -p /etc/apt/keyrings
curl -fsSL [https://download.docker.com/linux/ubuntu/gpg]
(https://download.docker.com/linux/ubuntu/gpg) | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
```

Step 2.2: Mount the Stable Official Package Channel

Bash

```
echo \
```

```
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
[https://download.docker.com/linux/ubuntu]
(https://download.docker.com/linux/ubuntu) \
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list >
/dev/null
```

Step 2.3: Execute Container Runtime Installation

Bash

```
sudo apt-get update -y
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-
plugin
```

Step 2.4: Privilege Abstraction for GitOps Runner Automation

By default, Docker binds to privileged root sockets. To enable headless automation through non-privileged agents (GitHub Action Run scripts), execute the socket group allocation routine:

Bash

```
# Enforce system docker network group initialization
```

```
sudo groupadd -f docker
```

```
# Inject your active administrative shell user into the permission matrix
```

```
sudo usermod -aG docker $USER
```

```
# Force reload system group permissions dynamically without breaking active SSH
sessions
```

```
newgrp docker
```

3. Kubernetes-in-Docker (KinD) Binary Provisioning

KinD handles orchestration nodes natively on standard virtualization hardware without deploying expensive Cloud Provider Controllers.

Step 3.1: Mount and Allocate Target Core Binaries

Bash

```
# Fetch the verified AMD64 architecture Linux binary compilation
```

```
curl -Lo ./kind [https://kind.sigs.k8s.io/dl/v0.20.0/kind-linux-amd64]
(https://kind.sigs.k8s.io/dl/v0.20.0/kind-linux-amd64)
```

```
# Grant execute boundaries onto the newly generated binary footprint
```

```
chmod +x ./kind
```

```
# Shift the operational block directly into your global shell PATH
```

```
sudo mv ./kind /usr/local/bin/kind
```

4. Kubectl CLI Command Utilities Setup

kubectl acts as the declarative API interface client used to evaluate, map, and roll out microservice structures inside the cluster namespace.

Step 4.1: Bind Stable Kubernetes Client Tooling

Bash

```
# Extract the production-approved stable release component asset
```

```
curl -LO "[https://dl.k8s.io/release/v1.28.0/bin/linux/amd64/kubectl]
(https://dl.k8s.io/release/v1.28.0/bin/linux/amd64/kubectl)"
```

```
# Establish executable operational capabilities on the target vector
```

```
chmod +x ./kubectl
```

```
# Move the administrative executor to the localized binary path tree
```

```
sudo mv ./kubectl /usr/local/bin/kubectl
```

5. Declarative Cluster Bootstrapping with Ingress Forwarding

To allow public traffic hitting your AWS EC2 Public IP address to seamlessly traverse down into your virtual inner cluster environment, map static proxy ingress routing rules.

Step 5.1: Create Ingress Configuration Schema Mapping (kind-config.yaml)

Bash

```
cat <<EOF> kind-config.yaml
```

```
kind: Cluster
```

```
apiVersion: kind.x-k8s.io/v1alpha4
```

```
nodes:
```

```
- role: control-plane
```

```
  extraPortMappings:
```

```
    - containerPort: 30080 # Target NodePort assigned in the K8s frontend spec
```

```
      hostPort: 8888 # Target network interface interface exposed to your AWS
```

```
EC2 SG
```

```
  protocol: TCP
```

```
EOF
```

Step 5.2: Execute Cluster Assembly Orchestration

Bash

```
kind create cluster --name skillpulse --config kind-config.yaml
```

6. Registering the GitHub Actions Self-Hosted Daemon Service

Establish a secure, persistent automation channel listening to repository code updates on GitHub.

Step 6.1: Initialize Isolated Workspace Directory

Bash

```
mkdir actions-runner && cd actions-runner
```

Step 6.2: Pull and Unpack Runner Source Assets

Bash

```
# Fetch targeted release distribution archive
```

```
curl -o actions-runner-linux-x64-2.311.0.tar.gz -L
```

```
[https://github.com/actions/runner/releases/download/v2.311.0/actions-runner-linux-x64-2.311.0.tar.gz]
```

```
(https://github.com/actions/runner/releases/download/v2.311.0/actions-runner-linux-x64-2.311.0.tar.gz)
```

```
# Extract compressed automation system structures
```

```
tar xzf ./actions-runner-linux-x64-2.311.0.tar.gz
```

Step 6.3: Bind Cryptographic Token Credentials

Note: Generate your exclusive repository runner registry token from: Repository Settings -> Actions -> Runners -> New self-hosted runner.

Bash

```
./config.sh --url [https://github.com/mayankrawall/github-actions-demo]
```

```
(https://github.com/mayankrawall/github-actions-demo) --token
```

```
<YOUR_SECRET_GENERATED_REGISTRATION_TOKEN>
```

Step 6.4: Enforce Native Persistence (Systemd Service Automation)

Bash

```
# Register the executable parameters inside standard Linux Systemd controls
```

```
sudo ./svc.sh install
```

```
# Initiate persistent background worker operations
```

```
sudo ./svc.sh start
```

7. Final System Integrity Validation

Run these checks to guarantee that all system infrastructure components are fully operational, secure, and ready for deployment jobs:

Bash

```
# 1. Inspect unprivileged rootless docker host capabilities
docker ps
```

```
# 2. Check localized virtualization cluster mapping state
kind get clusters
```

```
# 3. Assess running Kubernetes control plane node health
kubectl get nodes
```

```
# 4. Check systemic runner integration daemon active loops
sudo ./svc.sh status
```